# A Machine Learning and MLOps Framework for Customer Churn Prediction

B. V. S. Ganapathi Naidu(241FA18476)

Department of CSE-AIML Vignan's Foundation for Science, Technology and Research (Deemed to be University)
Vadlamudi, Andhra Pradesh, India

***Abstract - Customer churn is a significant concern for subscription-based businesses because the loss of existing customers can affect recurring revenue, customer lifetime value, and long-term business growth. This paper presents a machine learning workflow for predicting customer churn using a Telco customer dataset and an operational machine learning lifecycle. The prediction task is formulated as binary classification, with the Churn field serving as the target and customerID used as an identifier rather than a predictive feature. A recorded dataset load contained 7,344 rows and 21 columns. A Random Forest classifier achieved an accuracy of 0.7794, precision of 0.5630, recall of 0.7371, F1 score of 0.6384, and ROC-AUC of 0.8525. The experiment was tracked in MLflow under the churn_prediction_model_training_lab7 experiment. The model was registered as Telco_Churn_Production_Model, version 1, and promoted to the Production stage after satisfying a recall-based quality gate of at least 0.70. The workflow includes data loading, validation, preprocessing, training, evaluation, quality assessment, lifecycle management, and preparation of deployment artifacts. The results demonstrate how a customer churn model can be developed and managed through a reproducible MLOps workflow.***

***Index Terms - Classification, customer churn, machine learning, MLflow, MLOps, Random Forest.***

## I. INTRODUCTION

Customer churn occurs when a customer discontinues a service or subscription. For telecommunications and other subscription-based businesses, churn can reduce recurring revenue and increase the cost of replacing customers. Identifying customers who are more likely to leave can help an organization prioritize retention communication, service improvement, and customer support. Predictive analytics can support this process by estimating churn risk from historical customer and service information.

The project described in this paper treats churn prediction as a supervised binary classification problem. The target column is named Churn, and customerID is an identifier used to distinguish records. The model learns patterns in the available customer attributes and predicts whether a customer belongs to the churn class. The prediction is intended to support prioritization; it should not be interpreted as a guarantee that a particular customer will leave.

The project also emphasizes the machine learning operations lifecycle. In addition to training a classifier, the workflow includes data validation, preprocessing, experiment tracking, evaluation, a quality gate, model registry management, and preparation of deployment artifacts. This lifecycle-oriented structure makes it possible to track model versions and apply explicit conditions before promoting a model.

## II. DATASET AND PROBLEM FORMULATION

### *A. Dataset overview*

The recorded Telco churn dataset load contained 7,344 rows and 21 columns. The project configuration expected the raw dataset at data/raw/churn.csv. The target variable is Churn, while customerID is a record identifier. The dataset represents customer-level information and is used to learn the relationship between customer attributes and the observed churn outcome. The exact source URL, collection period, class distribution, and complete field descriptions were not included in the available run record and should be documented from the dataset provenance before publication.

### *B. Classification objective*

Let x represent the input attributes for a customer and y represent the churn label. The objective is to learn a classifier f(x) that predicts the churn class. In a binary formulation, the two outcomes represent customers who churned and customers who remained. The Churn field supplies the supervised label. The customerID field is used for identification and should not be treated as a behavioral predictor because it is an identifier rather than a customer service characteristic.

### *C. Data preparation*

The broader project workflow separates raw data loading, schema validation, preprocessing, and output validation. A recorded Lab 5 workflow validated the input

schema, fit and transformed training and test data, saved a preprocessing object as preprocessor.pkl, and produced a preprocessing summary report at artifacts/preprocessing_summary_report.json. The available run notes confirm that features were scaled and encoded, but do not specify the exact imputation strategy, categorical encoding configuration, feature list, or split parameters for the 7,344-row model run. These implementation details should be taken from the final configuration and report files when preparing a camera-ready version.

## III. MODEL DEVELOPMENT

### *A. Random Forest classifier*

The recorded model evaluation identifies Random Forest as the classifier used for the reported churn metrics. Random Forest is an ensemble learning method that combines predictions from multiple decision trees. Each tree is trained using a randomized sample of the training data and candidate features. Combining tree predictions can reduce the instability of an individual decision tree and can represent nonlinear relationships between customer attributes and churn.

For customer retention, the positive class is typically the churn class. This makes recall an important measure because a false negative represents a customer who churns but is not identified by the model. Precision is also important: low precision can cause retention teams to spend time contacting customers who would not have churned. The appropriate balance depends on the cost of outreach and the business impact of missed churn cases.

### *B. Evaluation measures*

Accuracy is the proportion of all predictions that are correct. Precision is the proportion of predicted churn cases that are actual churn cases. Recall is the proportion of actual churn cases correctly identified by the model. The F1 score is the harmonic mean of precision and recall. ROC-AUC measures the model's ability to rank churn cases above non-churn cases across different classification thresholds. Together, these metrics provide a more informative assessment than accuracy alone, particularly when the class distribution is uneven.

## IV. EXPERIMENTAL RESULTS

The recorded Random Forest evaluation produced an accuracy of 0.7794, precision of 0.5630, recall of 0.7371, F1 score of 0.6384, and ROC-AUC of 0.8525. The model therefore identified approximately 73.71% of the actual churn cases at the reported operating threshold. Precision of 0.5630 indicates that some customers predicted to churn did not belong to the actual churn class. This trade-off may be acceptable when the cost of missing a customer at risk is higher than the cost of additional retention outreach, but that decision requires business-specific cost analysis.

| Evaluation metric | Recorded value |
|---|---|
| Accuracy | 0.7794 |
| Precision | 0.5630 |
| Recall | 0.7371 |
| F1 score | 0.6384 |
| ROC-AUC | 0.8525 |

**TABLE I. RECORDED RANDOM FOREST CLASSIFICATION RESULTS.**

The recorded confusion matrix was [[859, 222], [102, 286]], using the run's reported class ordering. It contains 859 true negatives, 222 false positives, 102 false negatives, and 286 true positives when interpreted in the conventional binary classification layout. The 286 true positives and 102 false negatives are consistent with the reported churn recall of approximately 0.7371. The confusion matrix helps translate summary metrics into the types of errors that a retention workflow may experience.

The reported results belong to the recorded Random Forest experiment. The available project history does not provide a complete, directly comparable metric table for multiple candidate algorithms in this run. Consequently, this paper does not claim that Random Forest outperformed other algorithms or present unverified comparison values.

## V. MLOPS WORKFLOW AND EXPERIMENT TRACKING

### *A. Pipeline stages*

The churn project is organized as a sequence of modular workflow stages. The recorded full pipeline included data loading, raw data validation, preprocessing, model training and MLflow tracking, model evaluation, quality-gate assessment, and lifecycle management. Separating these stages makes it easier to identify failures and to rerun only the affected part of the workflow. The pipeline also produces reports and artifacts that support inspection of the model development process.

### *B. MLflow experiment tracking*

The training run was recorded under the MLflow experiment named churn_prediction_model_training_lab7. Experiment tracking associates the model run with its evaluation results and generated artifacts. This provides a record of the training activity and supports comparison and review of candidate model versions. The available run information identifies the experiment name and the registered production model, but does not provide a complete list of all hyperparameters or the source-control revision associated with the run.

### C. Quality gate

The project applied a recall-based quality gate requiring recall to be at least 0.70. The recorded Random Forest recall was 0.7371, so the model passed this condition. A recall threshold is appropriate when the workflow seeks to limit the number of customers who actually churn but are missed by the prediction system. Passing the gate means the model satisfied the configured metric condition; it does not by itself establish business profitability or guarantee performance on future customer populations.

### D. Model registry and lifecycle

The model was registered in MLflow under the name Telco_Churn_Production_Model, version 1, and promoted to the Production stage. The registry provides a named, versioned reference to the selected model and separates model promotion from the training script itself. A later deployment-preparation run reported downloading the Production model and saving deployment artifacts as deployment/model.pkl together with the matching preprocessor. This confirms preparation of local deployment artifacts; it should not be interpreted as proof that a publicly accessible production service was hosted.

## VI. DEPLOYMENT ARTIFACTS AND INFERENCE READINESS

The deployment-preparation workflow retrieves the registered Production model and its associated preprocessing artifact. Keeping the model and preprocessor together is necessary because inference data must undergo the same feature transformations that were applied during training. The recorded output paths were deployment/model.pkl and a matching preprocessor file. This artifact preparation provides a foundation for an inference application.

The available project record does not establish a fully verified public API, Docker image, or externally accessible deployment for the churn model. The documented achievement is model registration, Production promotion, and local deployment-artifact preparation. A complete serving deployment would additionally require an inference interface, request validation, health checks, dependency packaging, runtime tests, and hosting configuration.

## VII. TESTING, RELIABILITY, AND FAILURE HANDLING

The pipeline records validation and quality-control stages before model promotion. Raw-data validation checks whether the incoming data satisfies the expected schema, while output validation checks the transformed training and test artifacts. The quality gate then evaluates whether the model meets the configured recall requirement. These checks reduce the risk of registering a model that was trained from invalid data or does not meet the project's selected performance condition.

The project history also records a lifecycle deployment-update failure caused by an incorrect lifecycle report path in one pipeline execution. This illustrates why pipeline orchestration must verify artifact locations and stage outputs before continuing. A subsequent deployment-preparation run successfully retrieved the Production model and saved the model and preprocessor artifacts. These events should be distinguished: artifact preparation succeeded in the later run, while the earlier lifecycle-path issue remains a useful reliability lesson.

The available churn project records do not provide a confirmed automated API test count or a full inference test report. Such results should be added only after running and recording the relevant test suite.

## VIII. LIMITATIONS AND FUTURE WORK

The current report is limited by the experimental details present in the available project records. For full reproducibility, the final paper should include the exact dataset provenance, target class counts, feature names, missing-value treatment, categorical encoding, train-validation-test split, random seed, hyperparameter settings, software versions, and the precise model run identifier. The confusion matrix class ordering should also be confirmed against the evaluation report before final publication.

Future work can evaluate multiple algorithms under the same data split, tune the decision threshold according to retention costs, calibrate predicted probabilities, and validate performance on later time periods. Explainability methods can help retention teams understand the factors associated with individual predictions. Monitoring should track changes in customer behavior, feature distributions, recall, precision, and the rate of false alerts. A complete deployment can add an API, containerization, authentication, structured logging, and operational monitoring after the inference interface is implemented and tested.

## IX. CONCLUSION

This paper presented a customer churn prediction workflow built around a Random Forest classifier and an MLOps lifecycle. The recorded model achieved 0.7794 accuracy, 0.5630 precision, 0.7371 recall, 0.6384 F1 score, and 0.8525 ROC-AUC. It passed the configured recall threshold of 0.70, was registered as Telco_Churn_Production_Model version 1, and was promoted to Production. The project demonstrates how data validation, preprocessing, experiment tracking, evaluation, quality gates, and model registry management can be combined into a structured machine learning

workflow. Deployment artifacts were also prepared for the Production model. Further work is required to document all training details and to verify a complete externally accessible inference service.

## REFERENCES


[1] Scikit-learn Developers, “Scikit-learn User Guide,” scikit-learn documentation. Available: https://scikit-learn.org/stable/user_guide.html.

[2] MLflow, “MLflow Documentation,” MLflow documentation. Available: https://mlflow.org/docs/latest/.

[3] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[4] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.